UNIVERSITY OF WARWICK

Paper Details

Paper Code: CS2600

Paper Title: Algorithms

Exam Period: April 2026

Exam Rubric

Time allowed: 2 hours, plus 15 minutes reading time, during which notes may be made (on the question paper only) but you MUST NOT begin answering questions.

Exam Type: Written – Standard

Approved Calculators: Not Permitted

Instructions

Answer **ONE** question from Section A and **TWO** questions from Section B.

Indicate clearly which three questions you have answered. Among the questions attempted, only the first one in Section A and the first two in Section B (in their order of appearance in the answer book) will be marked.

Read carefully the instructions on the answer booklet and make sure that the particulars required are entered on each answer booklet.

The exam question paper must not be removed from the examination venue.

UNIVERSITY OF WARWICK

Section A Answer **ONE** question from this section.

1. This question is about complexity and reductions.

   (a) Let X, Y, Y', Z be variables representing some algorithmic problems.
   Let C_X, C_Y, C_Z be variables representing some complexity classes.

   Amy finds a polynomial reduction from X in C_X to Y in C_Y.
   Bill finds a polynomial reduction from Y', also in C_Y, to Z in C_Z.

   Is it the case that $X \leq_P Z$? [10]

   * * *

   (b) Here, let "graph" refer to an undirected simple graph (no duplicate edges or self loops).
   A path in a graph is *Hamiltonian* if it includes every vertex once each.

   Let HAMILTONIAN-PATH be the problem of determining whether a Hamiltonian path exists in a graph.
   Let HAMILTONIAN-CYCLE be the equivalent problem but for a cycle—that is, the path starts and ends on the same vertex, but repeats are otherwise not allowed.

   Prove that HAMILTONIAN-PATH $\equiv_P$ HAMILTONIAN-CYCLE. [20]

   (You can call the two problems H-P and H-C to cut down on writing, if you prefer!)

Total for Question 1: 30

2. This question is about divide-and-conquer algorithms and recurrence relations.

   (a) Alfie has designed a divide-and-conquer algorithm to find the maximum value in an array A of n integers, where n is a power of 2.

   The algorithm recursively finds the maximum value in the left half of the array and recursively finds the maximum value in the right half of the array. It then compares these two values and reports the larger one as the overall maximum. For a subarray of length 1, the algorithm returns the single element and performs no comparisons.

   Let $T(n)$ denote the number of element-to-element comparisons performed by the algorithm on an array of length n.

   Write a recurrence relation for $T(n)$, and solve it for the number of comparisons. [8]

   (b) Alfie vibe codes his algorithm in Java, but the result differs a bit from the plan.

   Instead of dividing the array into two halves, the program recursively computes the maximum of the first $\frac{3}{4}n$ elements of the array and the maximum of the last $\frac{3}{4}n$ elements of the array, so the two subarrays overlap. It then compares the two reported values and returns the larger one.

   Is Alfie's implementation correct, i.e., does it always return the maximum? Is it asymptotically optimal in terms of the running time? [10]

   (c) Alicia wants to use an algorithm to sort an array B of n integers, where n is a power of 3. Alicia decides to implement the MergeSort algorithm in Python, but makes a mistake in her code: her program recursively sorts the first $\frac{2}{3}n$ elements, and then it recursively sorts the final $\frac{1}{3}n$ elements. It then performs a standard two-way merge that takes time linear in the total size of inputs.

   Analyze the cost of Alicia's implementation using the recursion tree method. [12]

   Hint: Consider the depth of the recursion tree, and the maximum number of elements processed at any level of the tree. Note that the Master theorem does not apply here.

Total for Question 2: 30

Section B Answer **TWO** questions from this section.

3. This question is about multiplying multiple matrices together.

   (a) Let X be an $n \times \ell$ matrix and Y an $\ell \times m$ matrix. Their product $Z = XY$ is the $n \times m$ matrix defined by

   $$Z_{i,j} = \sum_{k=1}^{\ell} X_{i,k} Y_{k,j}.$$

   Describe a simple algorithm that directly follows this definition to compute Z, and analyze its running time. [8]

   (b) Let A, B, and C be matrices with dimensions $n_1 \times n_2$, $n_2 \times n_3$, and $n_3 \times n_4$, respectively. There are two ways to compute the product ABC: $(AB)C$ or $A(BC)$.

   Using the algorithm from part (a), give the running time of an algorithm that computes ABC by choosing the faster of these two options. [7]

   (c) Suppose we are given a long sequence of matrices $X_1 X_2 \cdots X_m$, where X_i has dimensions $n_i \times n_{i+1}$. Show that the number of different orderings of operations to compute this product is exponential in m. [8]

   Hint: Rather than try to count every possible option, it is enough to show that even a restricted subset of options is exponential in size. In particular, for each block of three consecutive matrices, consider the independent choice between

   $$(X_{3i+1}(X_{3i+2}X_{3i+3})) \quad \text{and} \quad ((X_{3i+1}X_{3i+2})X_{3i+3}).$$

   (d) We want to determine an optimal order of multiplication for the matrix chain $X_1 X_2 \cdots X_m$, where X_i has dimensions $n_i \times n_{i+1}$. The goal is to minimize the total number of scalar multiplications when using the algorithm from part (a) for each matrix multiplication.

   Give a polynomial-time algorithm to find an optimal order, and analyze its running time. [12]

   Hint: Use dynamic programming.

Total for Question 3: 35

4. This question is about matching problems.

   The University of Borwick runs dissertation projects. Let A, B, and C be sets of students, supervisors and topics respectively. The students are allowed to list possible supervisors, and supervisors list the possible topics they could supervise.

   Let a *project* be a triple $(a, b, c) \in A \times B \times C$. The project organiser is looking for the biggest possible valid set of projects. A set of projects is *valid* if students are only paired with supervisors they selected; supervisors are only supervising topics they selected; and each student, supervisor and topic appears in **at most one** project.

   Suppose we have $A = \{\text{Alfie, Alicia, Adam, Anna}\}$, $B = \{\text{Boris, Belinda, Billie}\}$, and $C = \{\text{Clustering, Classification, Compression, Coding}\}$. The list of possible supervisors for each student, S, and possible topics for each supervisor, T, are given below.

   $S =$

Student	Supervisor
Alfie	Boris
Alicia	Boris
Alicia	Billie
Adam	Belinda
Adam	Billie
Anna	Billie

   $T =$

Supervisor	Topic
Boris	Clustering
Boris	Classification
Belinda	Classification
Belinda	Compression
Billie	Compression
Billie	Coding

   (a) Find a valid set of projects of maximum size for the given A, B, C, S, T above. Why can there not be a larger valid set of projects? [5]

   (b) We could find a valid set of projects that is as big as possible by encoding the assignment constraints as a flow network and computing a maximum flow. Explain how. [10]

   (c) Explain why your answer from (b) will find the biggest possible valid set of projects. You may assume standard properties of maximum flow algorithms. [10]

   (d) The Computer Science department is considering a new model, where a list of *suitable* projects is published, given as a set $P' \subseteq A \times B \times C$. In the new model, topics can be reused, and each supervisor can supervise any number of students, but all of those students **must** study the same topic, and every student must have one project.

   Let GOOD-PROJECTS be the problem of determining, given sets A, B, C, P', whether a set $Q \subseteq P'$ exists satisfying the requirements. Show that GOOD-PROJECTS is NP-hard by reduction from 3-SAT. [10]

   Hint: Consider an instance with two topics, corresponding to 'true' and 'false'. Represent supervisors as 3-SAT variables and students as clauses.

Total for Question 4: 35

5. This question is about designing and analyzing algorithms for sparse matrix operations.

   A *super-sparse* $n \times n$ matrix is a matrix that has exactly one 1 in each row and each column, and is 0 everywhere else.

   It is convenient to represent a super-sparse matrix X by an array $A(X)$, where $A(x)[i]$ denotes the location of the 1 in row i.

   For example, the array $[1, 2, 4, 3]$ represents the super-sparse matrix $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$.

   (a) Let X be the super-sparse matrix represented by $[1, 2, 4, 3]$ and Y the super-sparse matrix represented by $[4, 3, 1, 2]$.

   Write the transpose of Y i.e., Y^T, and compute the matrix product (XY).

   Express both answers using regular matrix notation. [4]

   (b) Given a super-sparse $n \times n$ matrix X, describe an algorithm to compute its array representation $A(X)$ as efficiently as possible. Then, given an array $A(X)$ of length n, give an efficient algorithm to compute the corresponding super-sparse matrix X. Analyze the running time of each algorithm. [8]

   (c) Given a super-sparse $n \times n$ matrix in array form as $A(X)$, describe an algorithm to compute $A(X^T)$. Your solution should avoid materializing the matrix X and should run in $o(n^2)$ time. [6]

   (d) Prove that if X is an $n \times n$ super-sparse matrix and Y is an $n \times n$ super-sparse matrix, then their product (XY) is also an $n \times n$ super-sparse matrix. [7]

   (e) Given two $n \times n$ super-sparse matrices in array form as $A(X)$ and $A(Y)$, give an algorithm to compute $A(XY)$, the array representation of their product. The asymptotic complexity of your algorithm should be as low as possible. Prove the running time of your algorithm. [10]

Total for Question 5: 35

End